

ON THE UTILIZATION OF MATRIX-FREE TENSOR DECOMPOSITIONS IN THE APPROXIMATION OF ACCURATE TENSOR ALGORITHMS.

KARL PIERCE

Tensors and Contractions

Vector

$$A_i$$

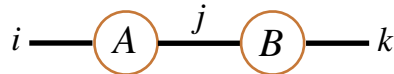
Matrix

$$A_{ij}$$

Higher-order tensor

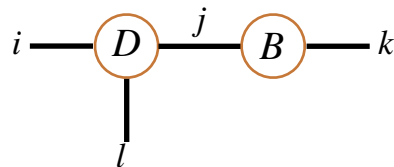
$$A_{ijk}$$

$$C_{ik} = \sum_j A_{ij} B_{jk}$$



$$\mathcal{O}(N^3)$$

$$F_{ikl} = \sum_j D_{ijl} B_{jk}$$



$$\mathcal{O}(N^4)$$

The curse of dimensionality

	Computational Scaling	Storage	Element of highest order tensor
CCSD	$\mathcal{O}(N^6)$	$\mathcal{O}(N^4)$	t_{ab}^{ij}
CCSDT	$\mathcal{O}(N^8)$	$\mathcal{O}(N^6)$	t_{abc}^{ijk}
CCSD(T)	$\mathcal{O}(N^7)$	$\mathcal{O}(N^4)$	t_{ab}^{ij}

Reformulating higher-order tensors into a set of lower-order tensors
can be used to break the curse of dimensionality

Tensor compression techniques

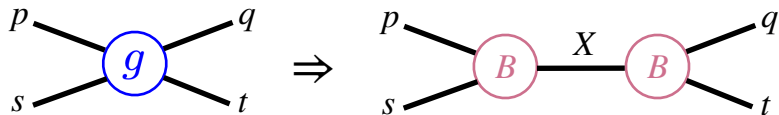
Matrix Decomposition

Singular Value
Decomposition

Cholesky
Decomposition

Linear Algebra Decomposition strategies

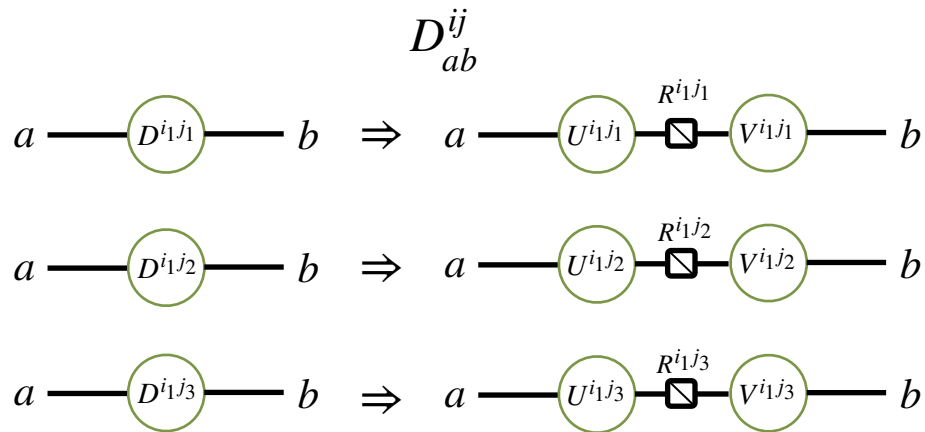
The density fitting approximation



$$g_{st}^{pq} \approx \sum_X B_s^{pX} B_t^{qX}$$

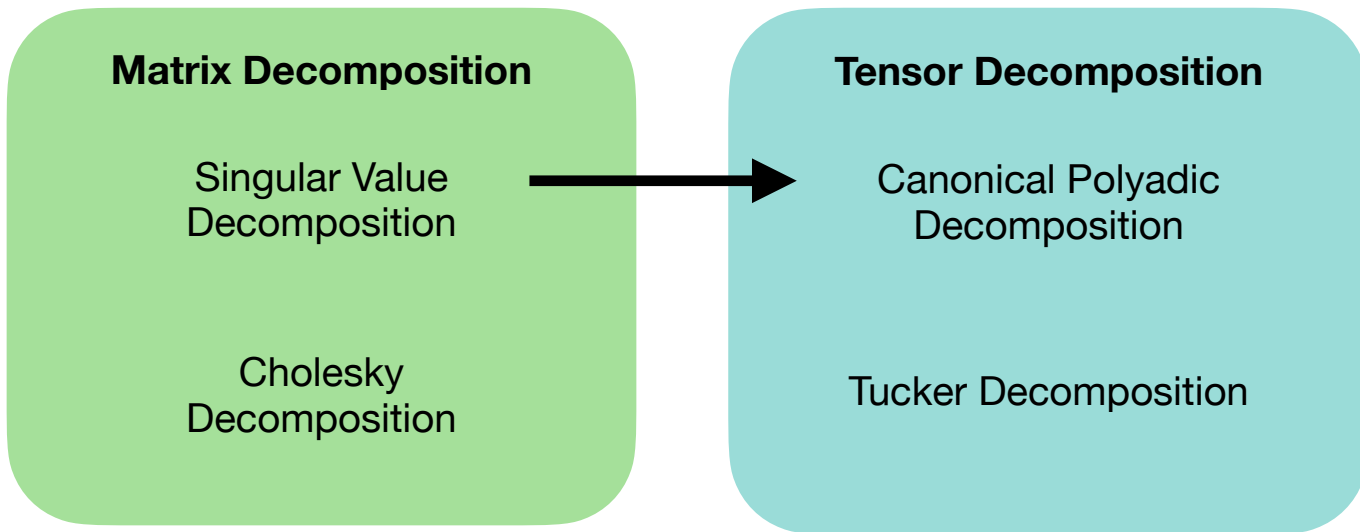
Analytically constructed with the Cholesky Decomposition

The pair natural orbital approach



Constructed via many pair-wise SVDs

Tensor compression techniques



Singular value decomposition (SVD)

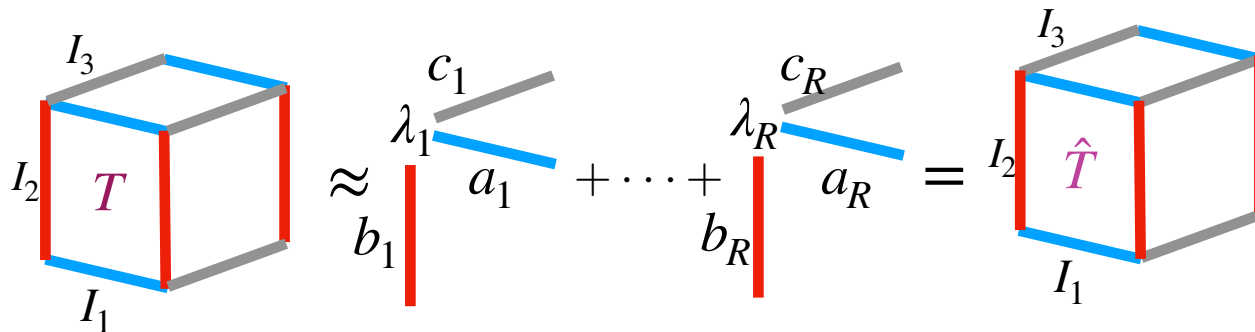
$$\begin{array}{c} I_2 \\ \boxed{T} \\ I_1 \end{array} = \begin{array}{c} \lambda_1 \text{---} a_1 \\ | \\ b_1 \end{array} + \cdots + \begin{array}{c} \lambda_R \text{---} a_R \\ | \\ b_R \end{array}$$

$$\begin{array}{c} I_2 \\ \boxed{T} \\ I_1 \end{array} = \begin{array}{c} I_2 \\ \boxed{} \\ I_1 \end{array} + \cdots + \begin{array}{c} I_2 \\ \boxed{} \\ I_1 \end{array}$$

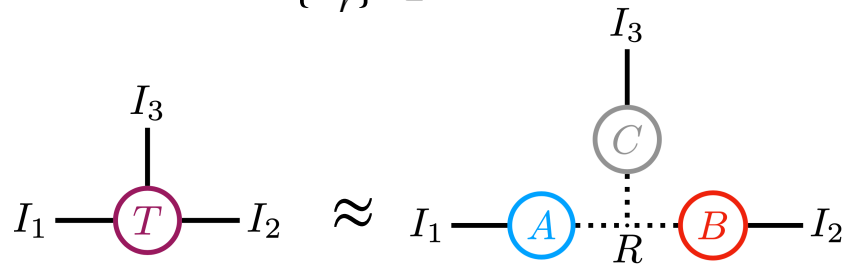
$$I_1 \text{---} \boxed{T} \text{---} I_2 = I_1 \text{---} \boxed{A} \cdots \boxed{B} \text{---} I_2$$

R

Canonical Polyadic Decomposition (CPD)



$$A = \{a_r\} \in \mathcal{R}^{I_1 \times R}$$



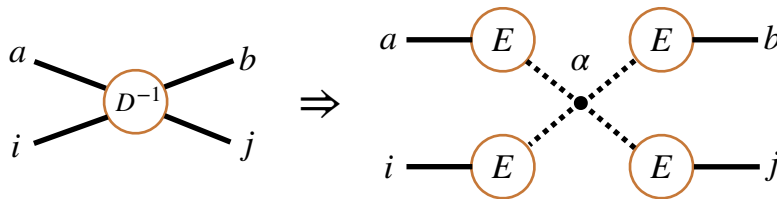
$$\mathcal{O}(I^3) \rightarrow \mathcal{O}(3IR)$$

An application of the CPD

$$D_{ij}^{ab} = (\epsilon_a + \epsilon_b - \epsilon_i - \epsilon_j) \quad \Rightarrow \quad [D^{-1}]_{ij}^{ab}$$

Laplace transform of the molecular energy denominator

$$\begin{aligned} \frac{1}{D_{ij}^{ab}} &\stackrel{\text{LT}}{=} \int_0^\infty e^{-D_{ij}^{ab}t} dt \approx \sum_{\alpha} w_{\alpha} e^{-D_{ij}^{ab}t_{\alpha}} \\ &= \sum_{\alpha} w_{\alpha} e^{-\epsilon_a t_{\alpha}} e^{-\epsilon_b t_{\alpha}} e^{\epsilon_i t_{\alpha}} e^{\epsilon_j t_{\alpha}} \end{aligned}$$



Analytic construction of the CPD.

Given $T \in \mathbb{R}^{I_a \times I_b \times I_c}$ we seek the CPD $\hat{T}$

$$\hat{T}_{abc} = \sum_r^{R_{\text{CP}}} A_{ar} B_{br} C_{cr}$$

Define an optimization problem

$$f(\hat{T}) = \min_{\hat{T}} \frac{1}{2} \|T - \hat{T}\|^2$$

Analytic optimization can be too memory and operationally intensive.

Routes to improve CPD optimizations

1

Approximate canonically
expensive loss function

$$T \approx XY$$

$$f(\hat{T}; T) \approx f(\hat{T}; XY)$$

2

Approximate canonically
expensive tensor network
contractions

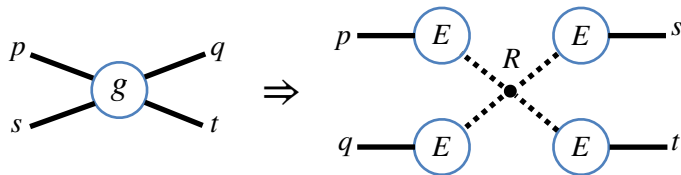
$$T = XY$$

$$f(\hat{T}; T) = f(\hat{T}; XY)$$

Approximately decompose the Coulomb integrals

Goal: decompose the two-particle (order-4) integral tensor

$$\hat{g}_{st}^{pq} = \sum_r^R P_r^p Q_r^q S_r^s T_r^t$$



$$f(\hat{g}) = \min_{\hat{g}} \frac{1}{2} \|g_{st}^{pq} - \sum_r^R P_r^p Q_r^q S_r^s T_r^t\|^2$$

Prohibitively large!!

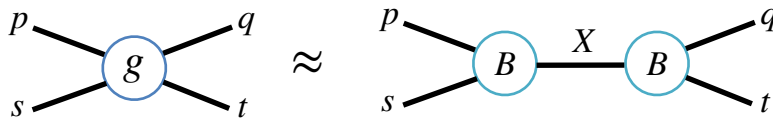
Storage: $\mathcal{O}(N^4)$

Scaling: $\mathcal{O}(N^5)$

Approximately decompose the Coulomb integrals

$$f(\hat{g}) = \min_{\hat{g}} \frac{1}{2} \| \hat{g}_{st}^{pq} - \sum_r^R P_r^p Q_r^q S_r^r T_r^r \|^2$$

The tensor G can be quickly decomposed via the DF approximation



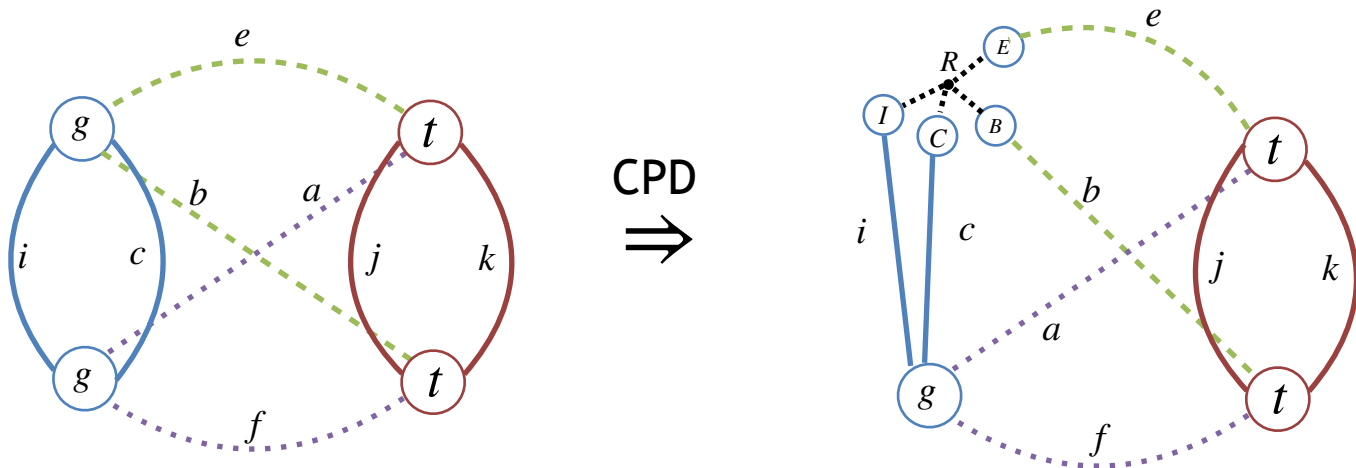
$$f(\hat{g}) \approx \min_{\hat{g}} \frac{1}{2} \left\| \sum_X B_s^{pX} B_t^{qX} - \sum_r^R P_r^p Q_r^q S_r^r T_r^r \right\|^2$$

Storage: $\mathcal{O}(N^3)$

Scaling: $\mathcal{O}(N^4)$

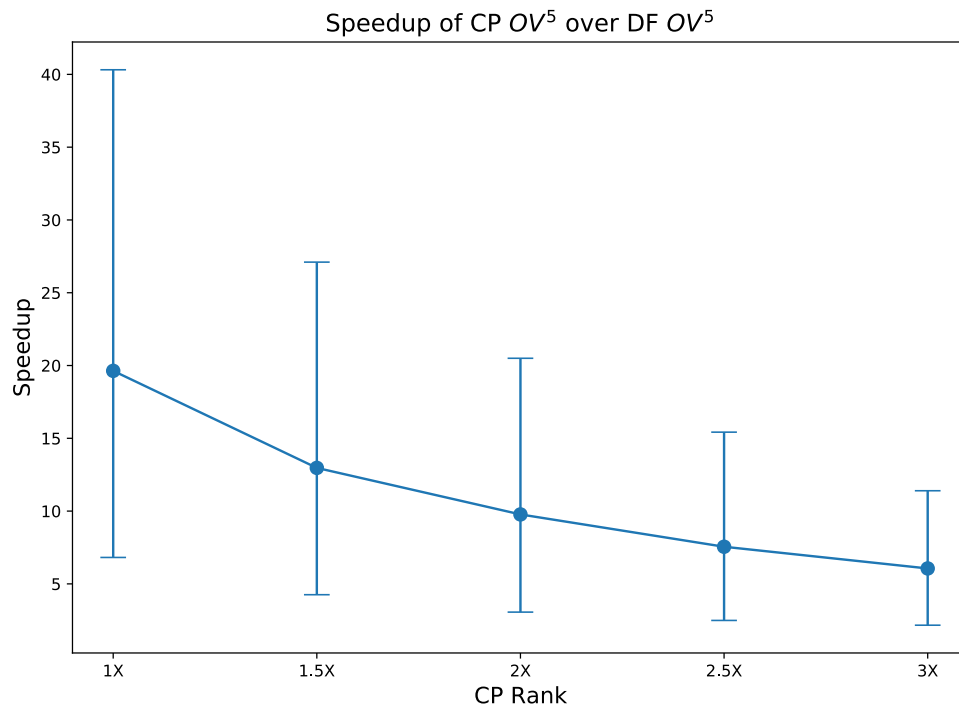
Application of CPD approximated G

The Laplace Transformed perturbative triples correction



$$E_{\text{OV}^5} = \sum_{ijkabcef} g_{bc}^{ie} g_{ac}^{if} t_{ea}^{jk} t_{bf}^{jk}$$

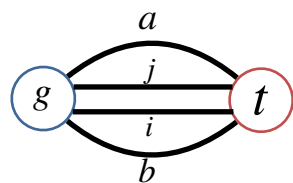
Result



1 - 12 water molecules in
TIP4P geometry
cc-pVDZ-F12/aug-cc-pVDZ-RI
Bars represent max/min values

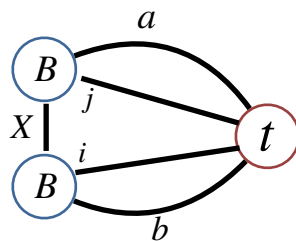
Are there more applications for the CPD?

$$E_{\text{MP2}_K} = \sum_{abij} t_{ij}^{ab} g_{ij}^{ba}$$



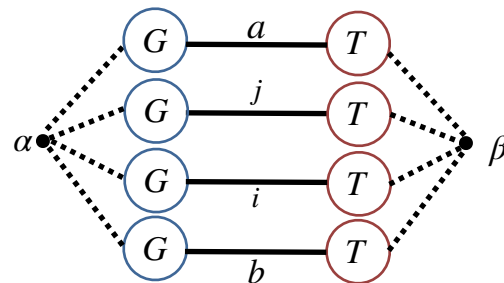
Scaling: $\mathcal{O}(N^4)$

$$E_{\text{MP2}_K} = \sum_{abij} t_{ij}^{ab} B_i^{bX} B_j^{aX}$$



$\mathcal{O}(N^4)$

$$E_{\text{MP2}_K} = \sum_{\alpha\beta} (G_{\alpha}^a T_{\beta}^a) (G_{\alpha}^b T_{\beta}^b) (G_{\alpha}^i T_{\beta}^i) (G_{\alpha}^j T_{\beta}^j)$$



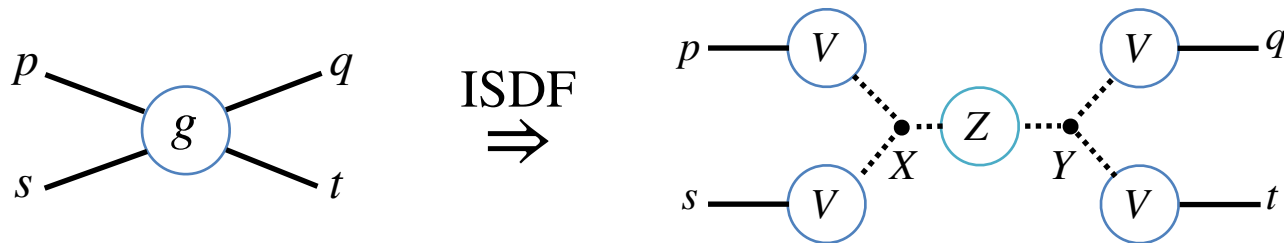
$\mathcal{O}(N^3)$

Unfortunately, forming these CPD approximations scale as $\mathcal{O}(N^4)$

Low scaling electron structure methods

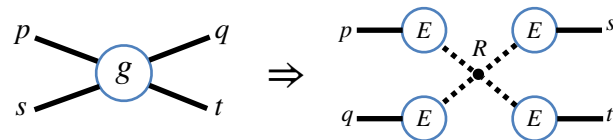
The interpolative decomposition has recently been leveraged to form a low-rank TEI tensor in $\mathcal{O}(N^3)$ time.

$$g_{ij}^{ab} \stackrel{\text{ISDF}}{\approx} \sum_{PQ} V_P^a V_i^P Z_Q^P V_Q^b V_j^Q$$

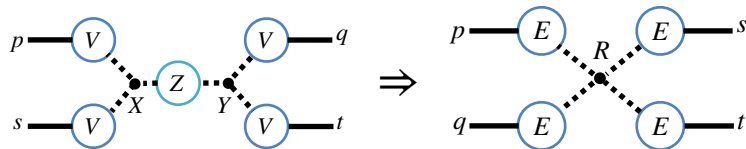


ISDF-CPD optimization problem

$$f(\hat{g}) = \min_{\hat{g}} \frac{1}{2} \|g_{st}^{pq} - \sum_r^R P_r^p Q_r^q S_r^s T_r^t\|^2$$



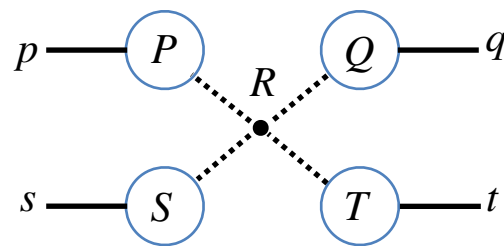
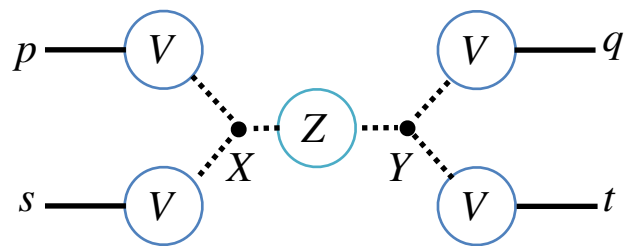
$$f(\hat{g}) \approx \min_{\hat{g}} \frac{1}{2} \left\| \sum_{XY} V_X^a V_i^X Z_Y^X V_j^b V_j^Y - \sum_r^R P_r^p Q_r^q S_r^s T_r^t \right\|^2$$



Storage: $\mathcal{O}(N^2)$

Scaling: $\mathcal{O}(N^3)$

Aside: The ISDF form is a CPD



$$g_{ij}^{ab} \stackrel{\text{ISDF}}{\approx} \sum_{XY} V_X^a V_i^X Z_Y^X V_Y^b V_j^Y$$

$$r = XY$$

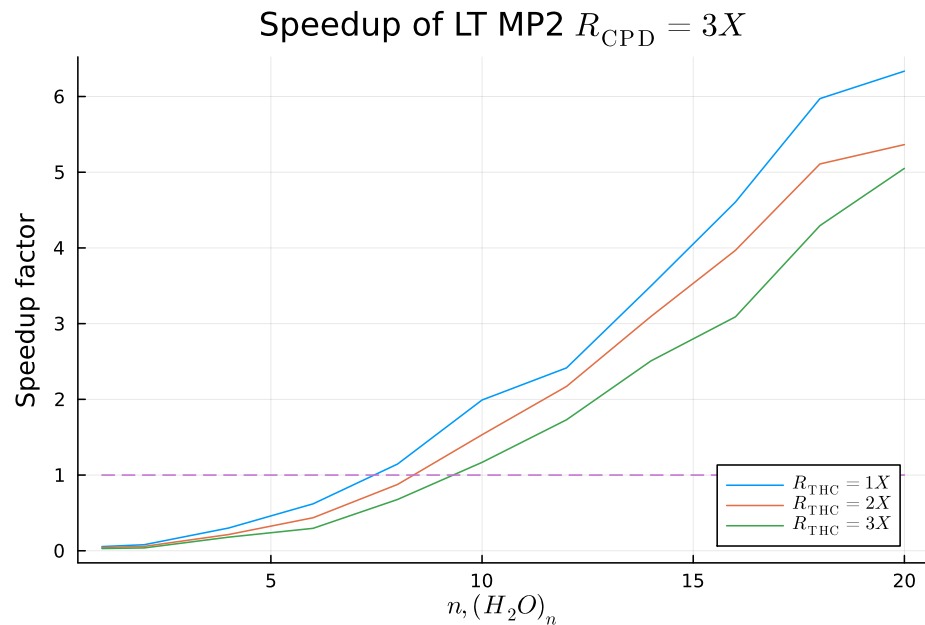
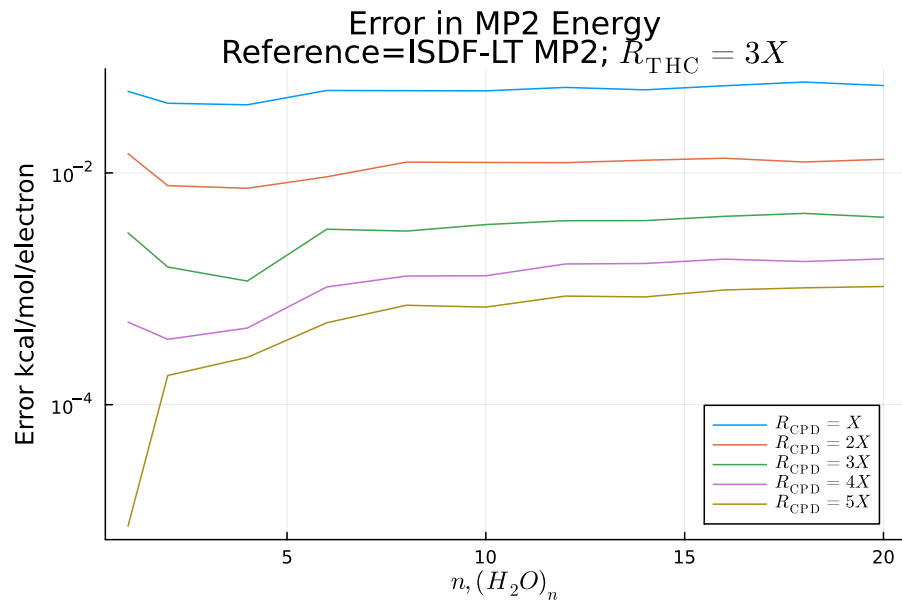
$$\lambda_r = Z_{XY}$$

$$P_r^a = V_X^a \otimes \mathbf{I}_Y$$

$$Q_r^b = V_Y^b \otimes \mathbf{I}_X$$

$$\hat{g}_{ij}^{ab} = \sum_r^R \lambda_r P_r^p Q_r^q S_r^s T_r^t$$

CPD-ISDF-LT MP2 Results



Recap

- Computing an analytic decomposition of a target tensor can be time and memory intensive
- In some cases a high-rank tensor can be quickly and effectively approximated in some other low rank format.
- My research leverages these low-rank forms to quickly re-optimize the tensor into a more useful representation.

Routes to improve CPD optimizations

1

Approximate canonically
expensive loss function

$$T \approx XY$$

$$f(\hat{T}; T) \approx f(\hat{T}; XY)$$

2

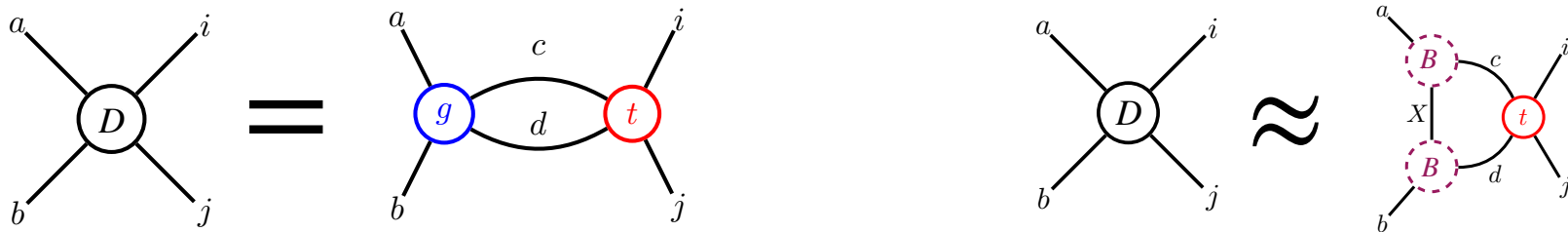
Approximate canonically
expensive tensor network
contractions

$$T = XY$$

$$f(\hat{T}; T) = f(\hat{T}; XY)$$

It is standard to approximate tensors in a network

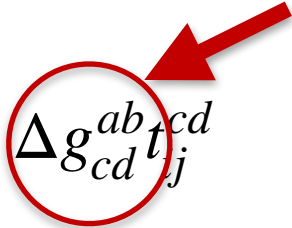
$$D_{ij}^{ab} = \sum_{cd} g_{cd}^{ab} t_{ij}^{cd} \xrightarrow{\text{Density Fitting Approximation}} D_{ij}^{ab} \approx \sum_{cdX} B_c^{aX} B_d^{bX} t_{ij}^{cd}$$

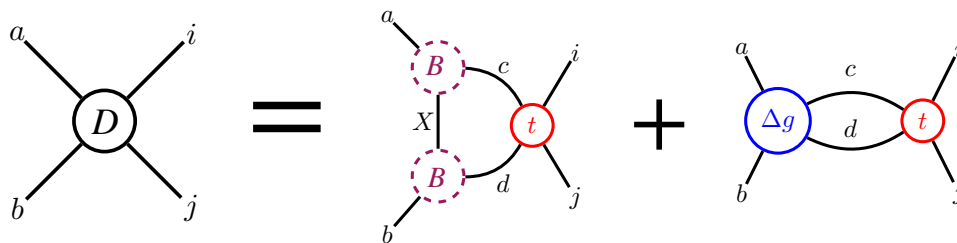


Error associated with approximated tensors in a network is amplified.

$$D_{ij}^{ab} = \sum_{cdX} B_c^{aX} B_d^{bX} t_{ij}^{cd} + \sum_{cdX} \Delta g_{cd}^{ab} t_{ij}^{cd}$$

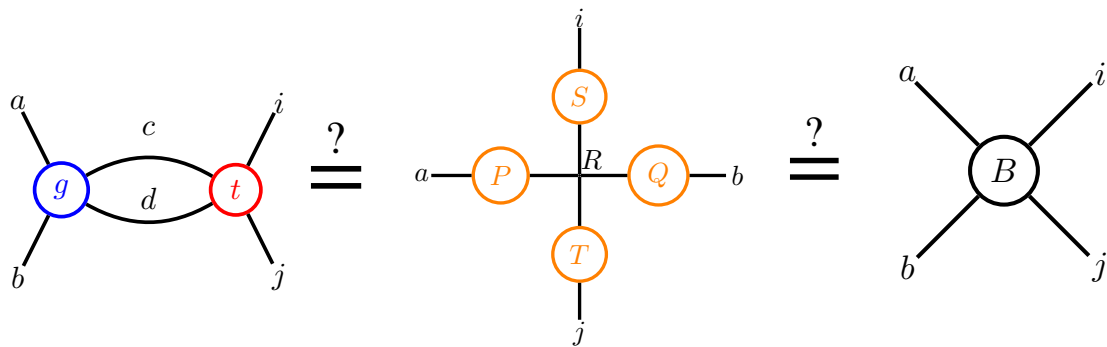
DF approximation error





Error propagation is exacerbated by size of network, magnitude of tensors,...

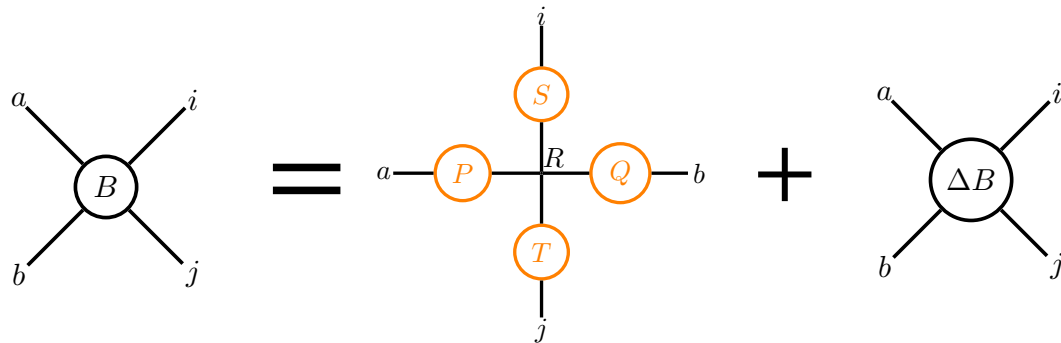
Is it possible to approximate the tensor contraction as a network?



$$f(\hat{B}) = \min_{\hat{B}} \frac{1}{2} \left\| \sum_{cd} g_{cd}^{ab} t_{ij}^{cd} - \hat{B}_{ij}^{ab} \right\|_2^2$$

$$\hat{B}_{ij}^{ab}(x) = \sum_r^{R_{cp}} P_r^a Q_r^b S_i^r T_j^r$$

Is it possible to approximate the tensor contraction as a network?



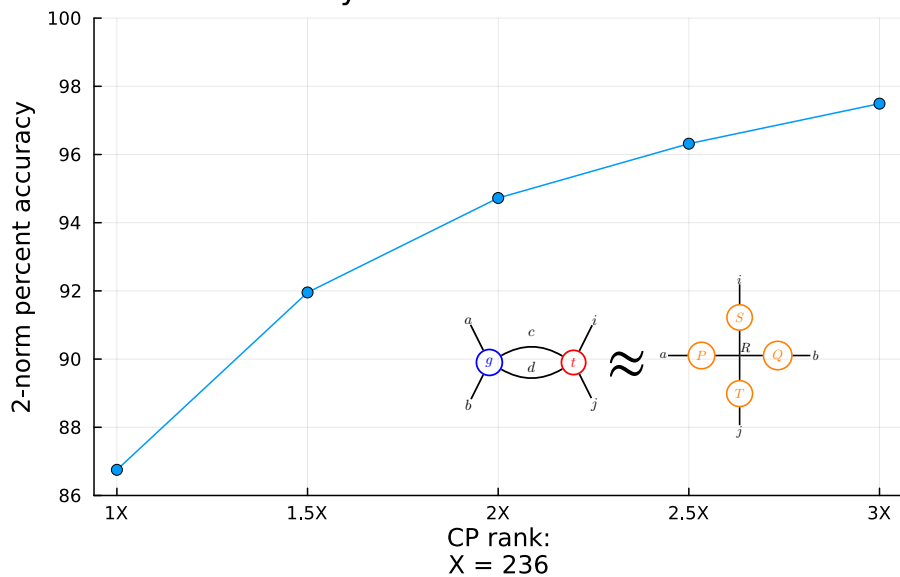
$$B_{ij}^{ab} = \hat{B}_{ij}^{ab} + \Delta_{CPD} B_{ij}^{ab}$$

$\Delta_{CPD} B$ is directly controllable via the CPD

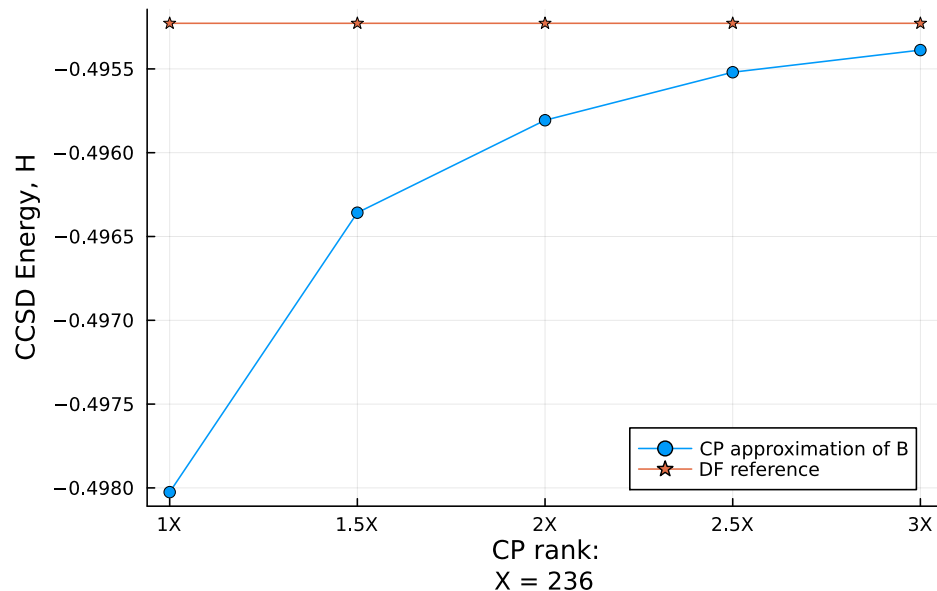
CP approximation of PPL tensor network

$(H_2O)_2$ TIP4P geometry
DZ-F12/aug-DZ-RI

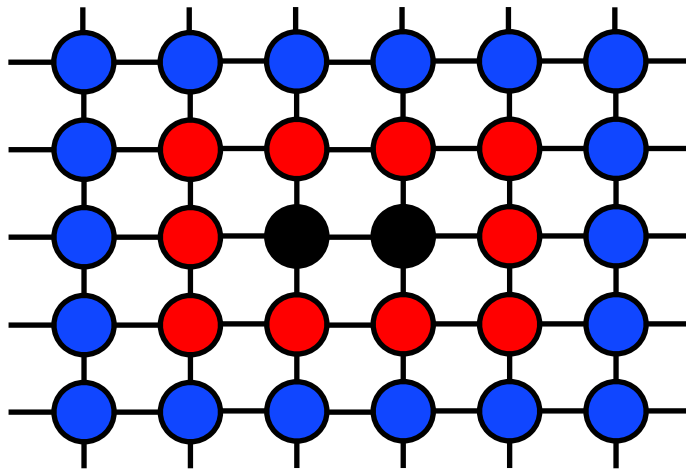
Accuracy of the CPD of the B tensor



CPD approximated CCSD Energy

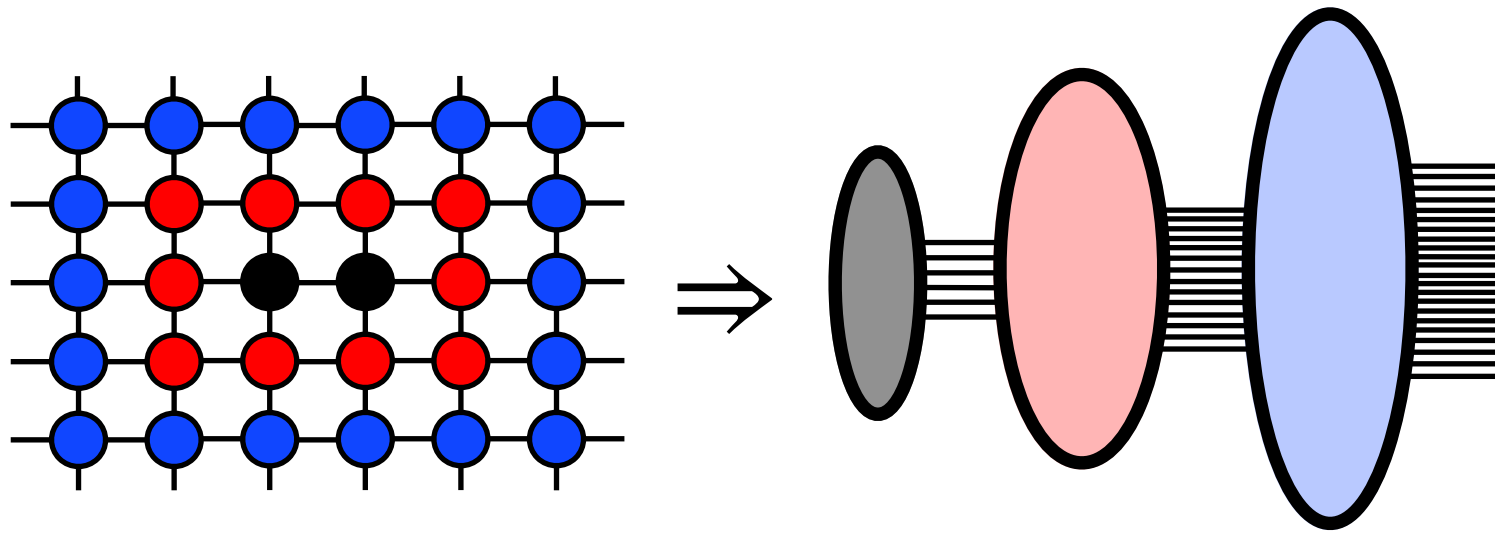


CP approximation of Lattice models



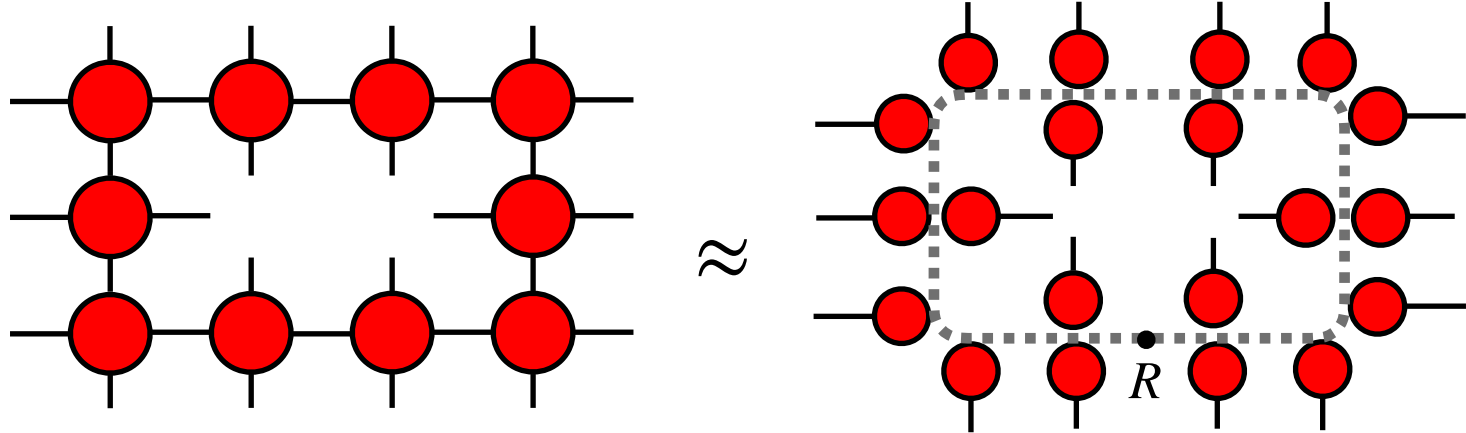
Two-dimensional tensor-networks can represent Ising partition functions, quantum circuits and beyond

CP approximation of Lattice models

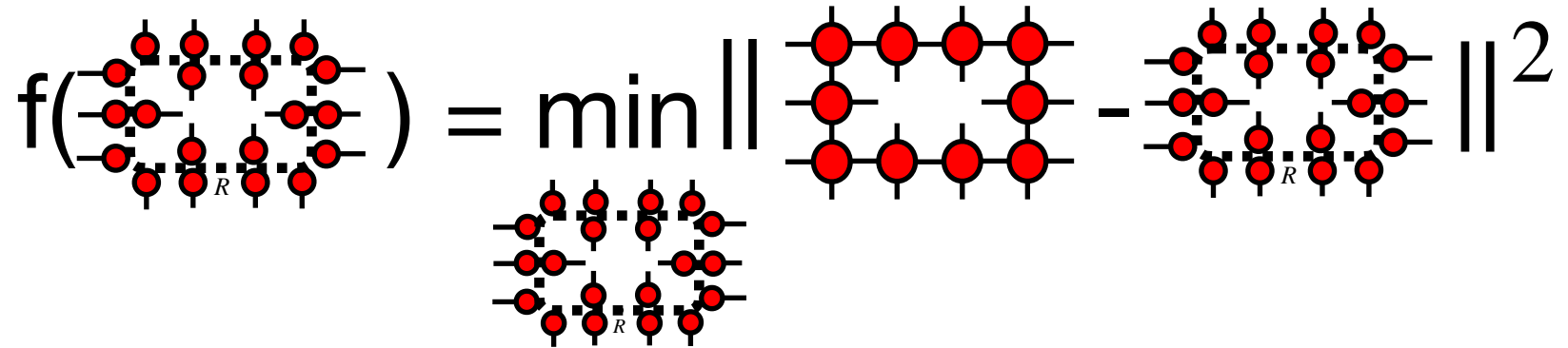


Contracting these models exactly becomes practically impossible very quickly

CP approximation of Lattice networks

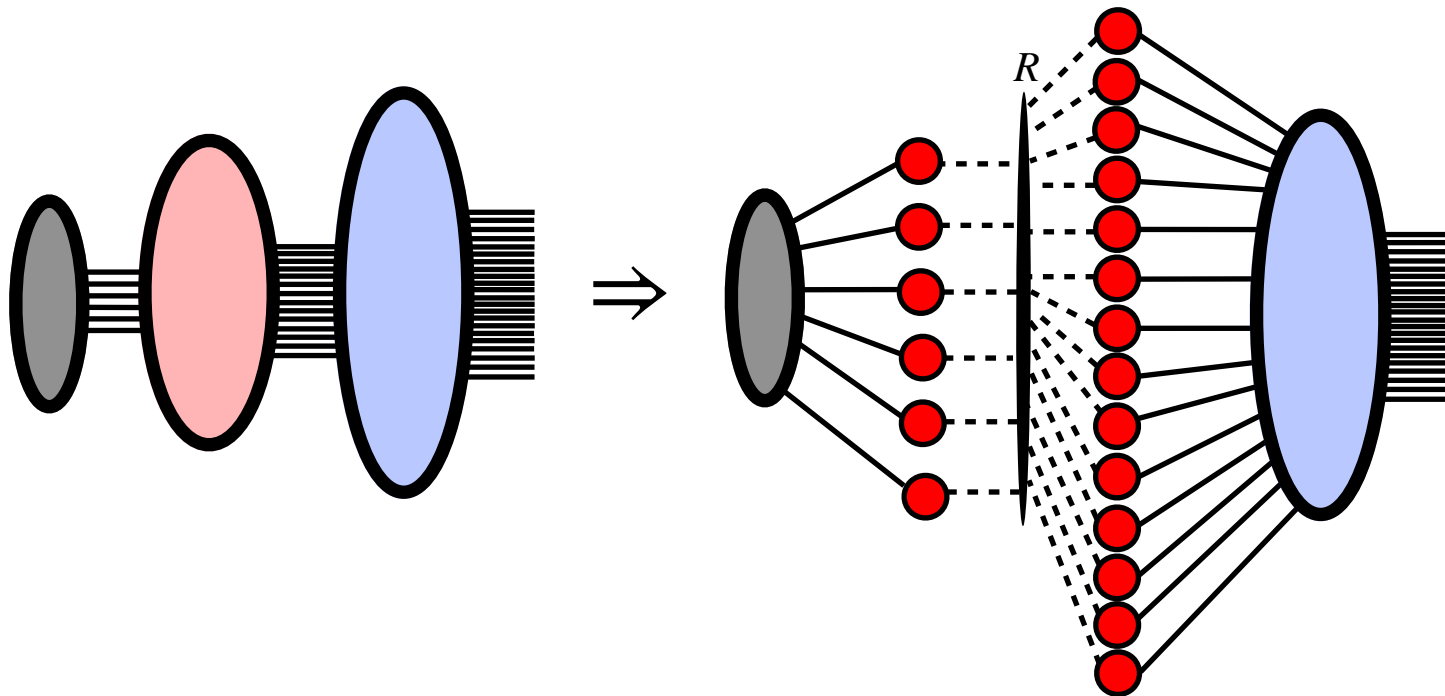


CP approximation of Lattice networks

$$f(\text{Lattice}) = \min_{\text{Lattice}} \left\| \begin{array}{c} \text{Lattice} \\ \text{Lattice} \end{array} - \text{Lattice} \right\|^2$$


The scaling of this optimization is constant with loop size

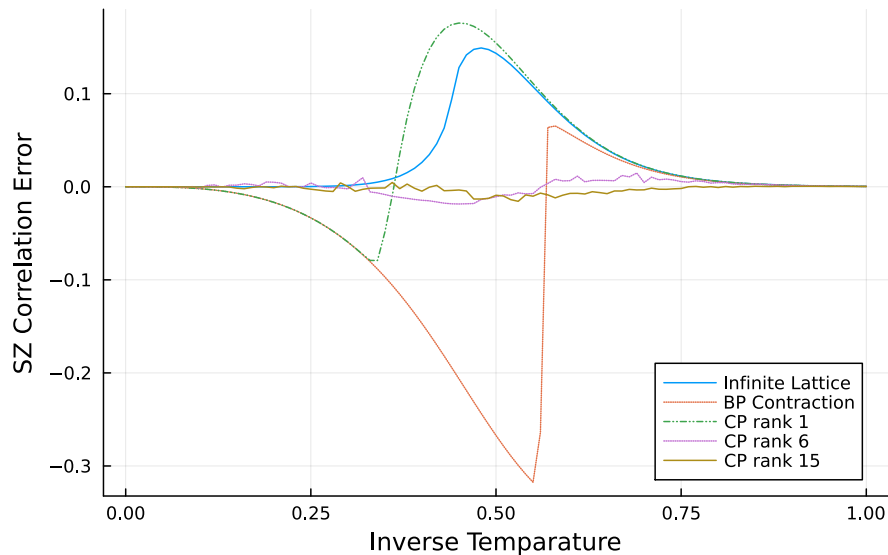
CP approximation of Lattice networks



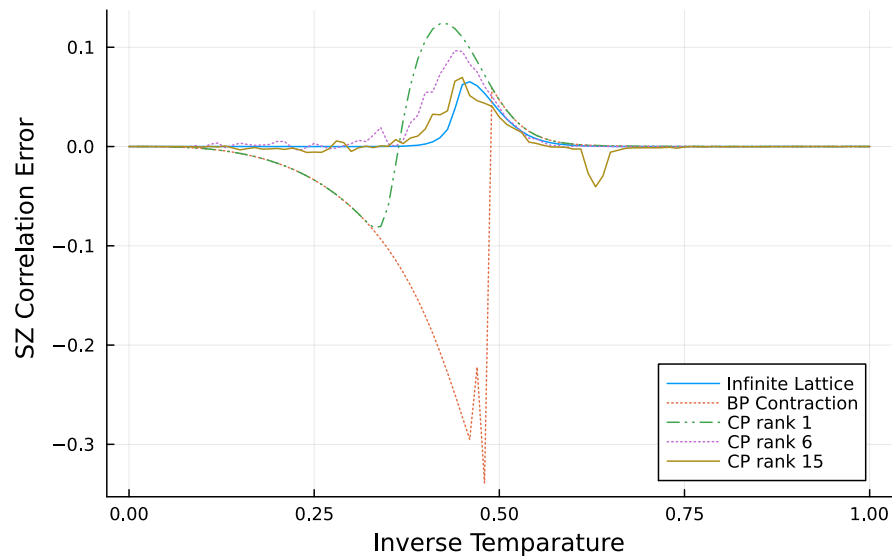
Decompose each matrix node simultaneously and independently

CP approximation of Lattice networks

CPD contraction of 2D network, 6x5 grid



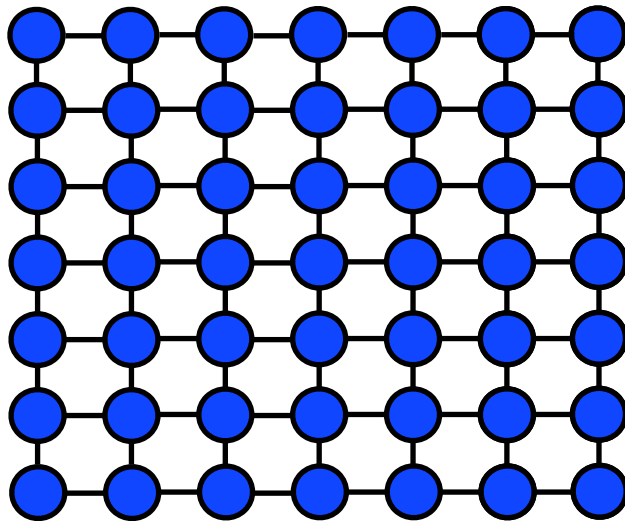
CPD contraction of 2D network, 14x13 grid



ITensor makes decomposing networks easy

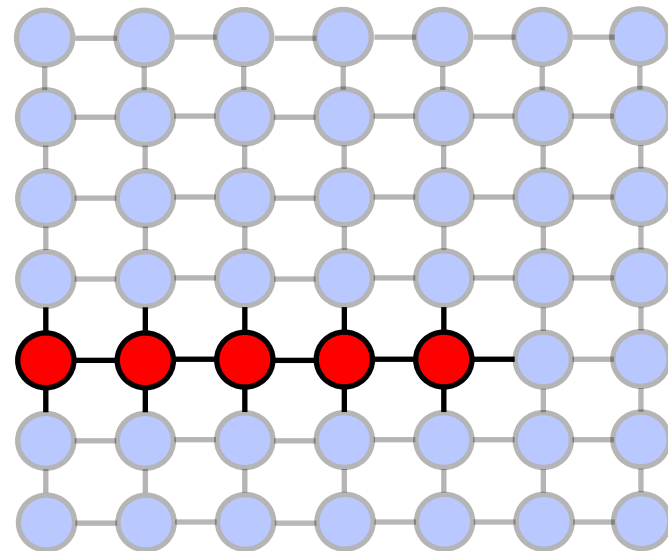
```
### Set up A 7 x 7 Grid ####
nx=7
ny=7
grid = named_grid((nx, ny))
indices = IndsNetwork(grid; link_space = 2)

### Fill the 7 x 7 grid with the Ising partition function ###
network = ising_network(Float64, indices, 0.4)
```



ITensorCPD

```
### Grab a subgraph of the network ###  
line_subtn = subgraph(network, ((5,1), (5,2), (5,3), (5,4), (5,5)))  
  
### Set up the CP problem ####  
r = Index(20, "CP_rank")  
n_linesubtn = norm_of_loop(line_subtn)  
fit = ITensorCPD.FitCheck(1e-3, 100, n_linesubtn)  
  
### Create a CP initial guess and optimize ###  
initial_guess = ITensorCPD.random_CPD_ITensorNetwork(line_subtn, r);  
@time cpopt = ITensorCPD.als_optimize(initial_guess, r, fit);
```



rank	iter	rel. accuracy	Δ accuracy
20	17	0.90255810291323	0.002330653462623067
20	18	0.9046042233858163	0.0020461204725862325
20	19	0.9058230185853863	0.001218795199570022
20	20	0.9066528771134078	0.000829858528021532
20	21	0.9072858251931635	0.0006329480797556686

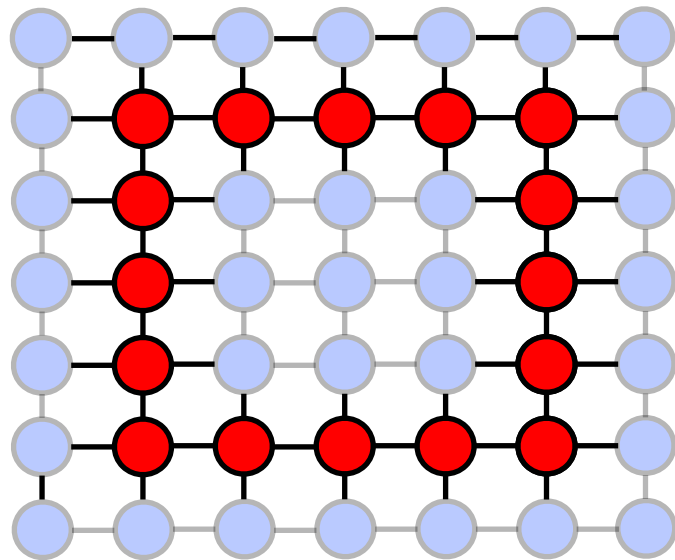
Completed in: 0.179472 seconds

ITensorCPD

```
### Grab a ring of tensors ###
ring_subtn = subgraph(network, ring_inds(2,7,7))

### Set up the CP problem ###
r = Index(300, "CP_rank")
n_ringsubtn = norm_of_loop(ring_subtn)
fit = ITensorCPD.FitCheck(1e-2, 100, n_ringsubtn)

### Create a CP initial guess and optimize ####
initial_guess = ITensorCPD.random_CPD_ITensorNetwork(ring_subtn, r);
@time cpopt = ITensorCPD.als_optimize(initial_guess, r, fit);
```



rank	iter	rel. accuracy	Δ accuracy
300	4	0.5383712709156414	0.020869993892039584
300	5	0.549899819840876	0.011528548925234583
300	6	0.5576682383035914	0.007768418462715365
300	7	0.5634688567447901	0.005800618441198724

Completed in: 6.001132 seconds

Recap

- It is possible to replace expensive contractions with matrix-free tensor approximations.
- Optimizing a full tensor network eliminates the propagation of error associated approximating nodes of a network.
- We can use a matrix-free optimization to approximately contract very high order tensors quickly.

Future work

- Studying element-wise sparsity preserving CPD optimization strategies.
- Introducing randomized algorithms to reduce optimization costs.
- Developing novel and low-scaling, $\mathcal{O}(N^3)$, quantum chemistry algorithms (i.e. rank-reduced coupled cluster methods)
- Find new problems to use these techniques!

